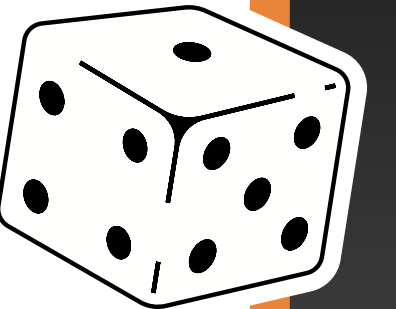




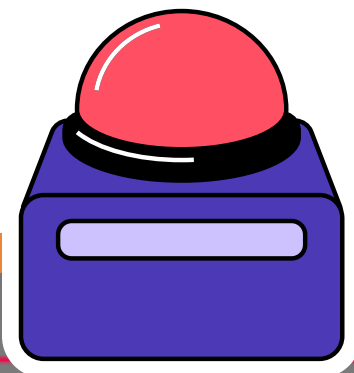
ACCESS MODIFIERS, SETTER & GETTER, FINAL CLASSES

ALGORITHMS & OBJECT-ORIENTED PROGRAMMING

WEEK 13

JOSEPH ANDREAS

FOR INTERNAL USE ONLY



AGENDA

1. ACCESS MODIFIERS

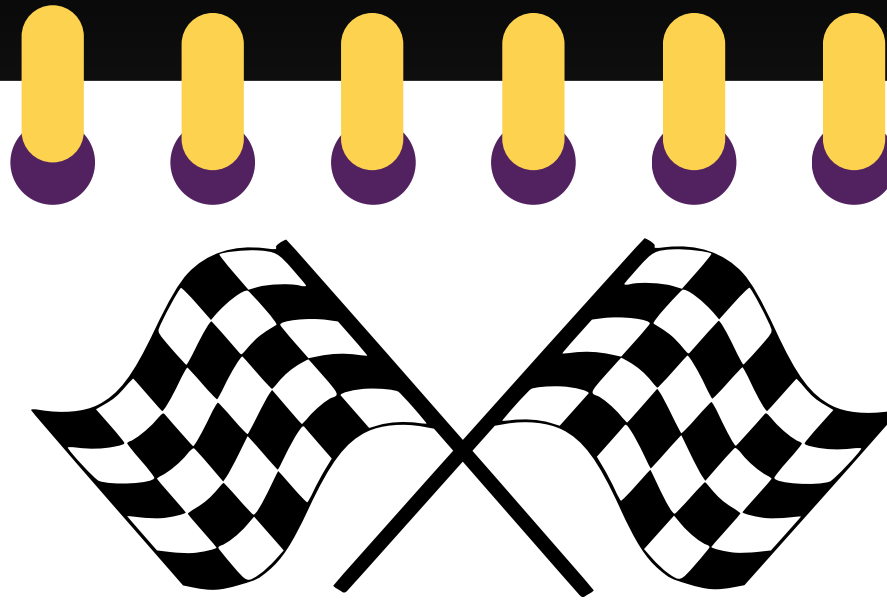
2. SETTER & GETTER

3. FINAL CLASSES

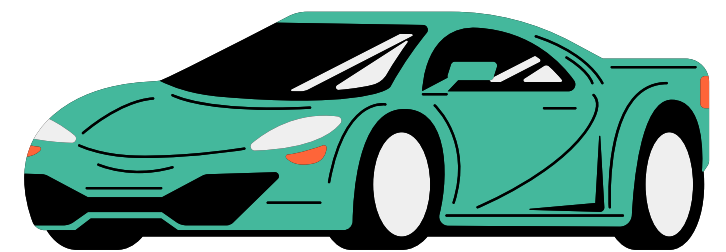
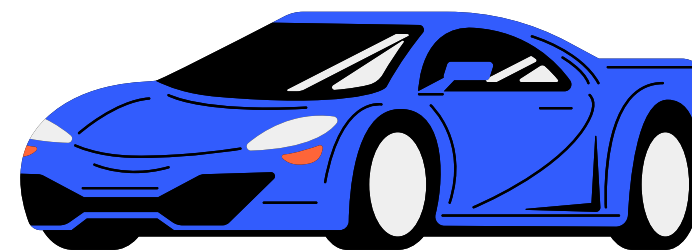
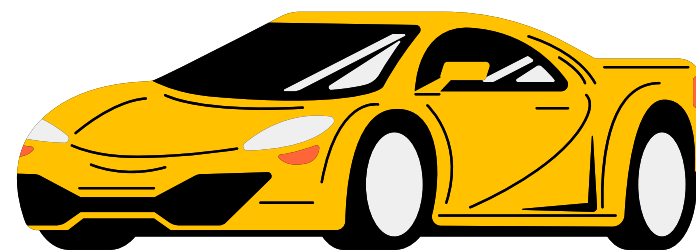
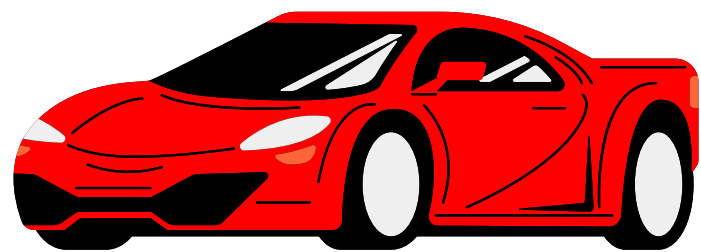


START





ACCESS MODIFIERS



ACCESS? MODIFY? MODIFY YOUR ACCESS?

- Access Modifiers: Controls which parts of your program can access a class, variable, or method
- There are three common access modifiers, which are:
 - Public: Open to all, can be accessed by any class, any package (everyone can see it)
 - Private: Hidden from others, can only be accessed inside the same class (only I can see it)
 - Default (no keyword): Package-level access, can be only accessed inside the same package (only classes in the same neighborhood {package} can see it)

PRIVATE VARIABLE, WHEN?

- When you want to protect the value from being changed directly
- When the variable represents sensitive or important data
- When changing the variable freely can break the object
- When you want to preserve the class design

WHY ACCESS MODIFIERS ARE IMPORTANT?

- Protect Data: Limits who can access variables and methods. Private keeps sensitive data hidden and prevents outside code from changing values accidentally → Also improves security
- Control Interaction with Other Classes: Decide what parts of the class are visible to other classes → Prevent misuse
- Easier to Maintain Code: By hiding internal details, how a class works internally can be changed without breaking other classes (e.g.: You can change your algorithm inside the class but the public methods stay the same so other parts of the program still work)
- Help Organize Large Projects: Create a clear boundary between components as many developers work on different modules and access modifiers help define which classes belong together

EXAMPLE OF ACCESS MODIFIERS

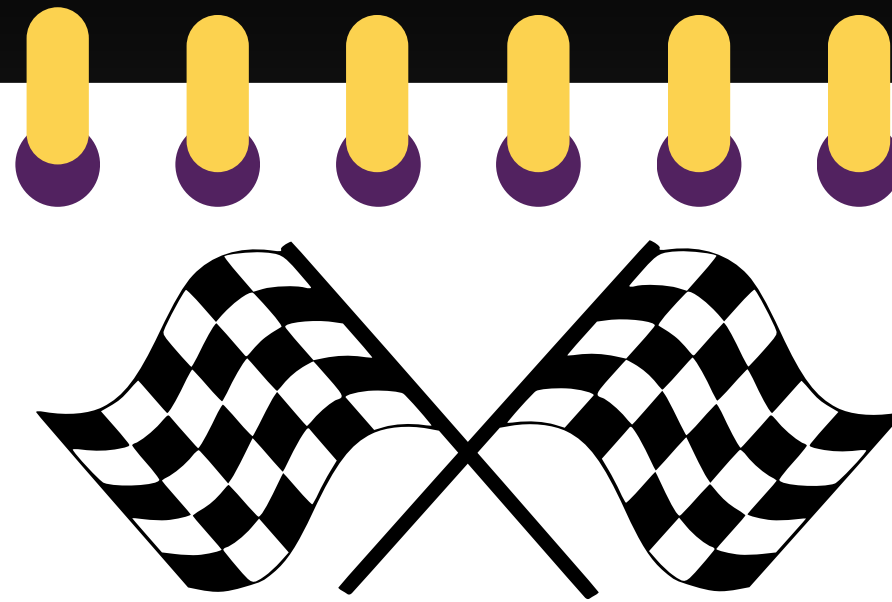
```
class Person {  
    public String name;  
    private int age;  
    String address;  
  
    public void showInfo() {  
        System.out.println("Name: " +  
name); → allowed  
        System.out.println("Age: " +  
age); → allowed  
        System.out.println("Address: " +  
address); → allowed  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Person p = new Person();  
  
        p.name = "Alice"; → public, OK  
        p.address = "Bangkok"; → default,  
OK (same package)  
        // p.age = 25; → private, NOT  
ALLOWED outside the class  
  
        p.showInfo();  
    }  
}
```

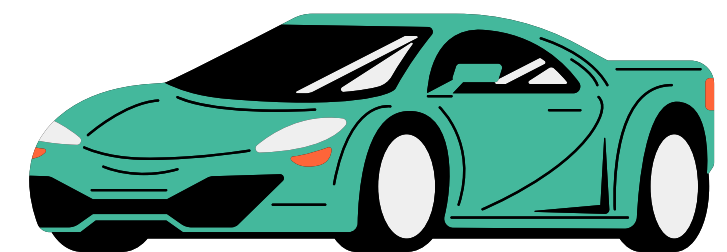
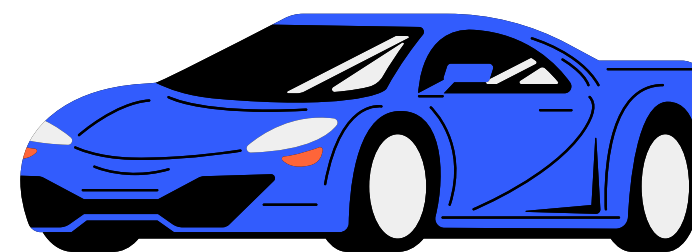
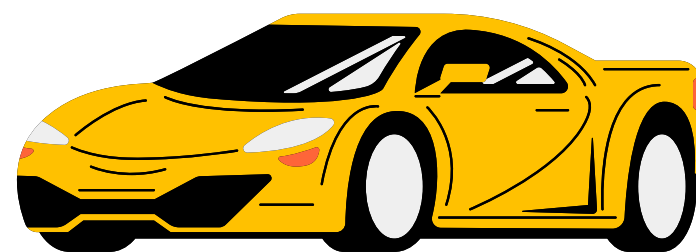
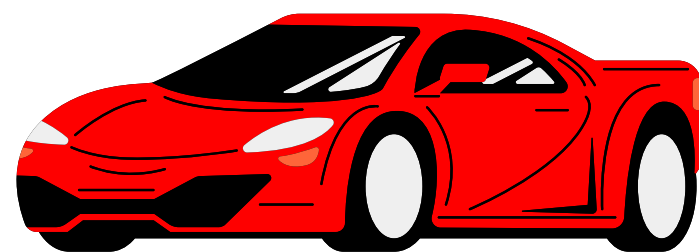
`name` is public, so anyone can access it

`age` is private, so it can only be accessed in
it's own class

`address` is default, so it can be accessed
within the same package



SETTER & GETTER



SETTER & GETTER METHODS

- Setter & Getter: Methods that allow controlled access to private variables
- Why Use Them?
 - Protect data (encapsulation)
 - Validate values before setting
 - Avoid exposing fields directly
- Formula
 - Setter → Updates the value
 - Getter → Returns the value

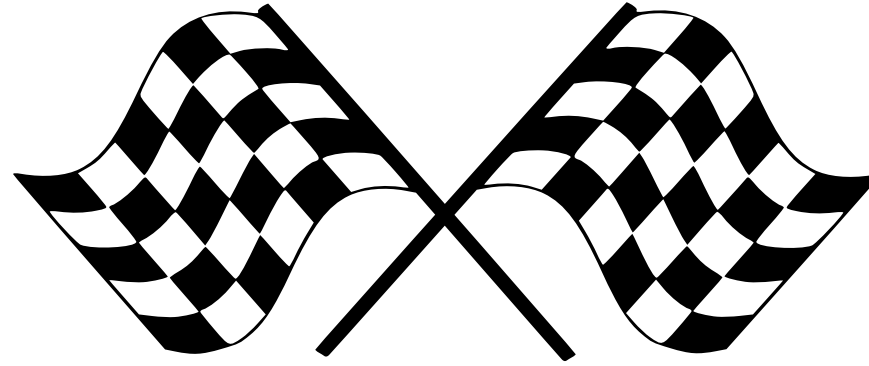
EXAMPLE OF SETTER GETTER

```
class Person {  
    public String name;  
    private int age;  
    String address;  
  
    public void showInfo() {  
        System.out.println("Name: " + name); → allowed  
        System.out.println("Age: " + age); → allowed  
        System.out.println("Address: " + address); →  
allowed  
    }  
  
    public void setAge(int age){ this.age = age;}  
    public int getAge(){return age;}  
}
```

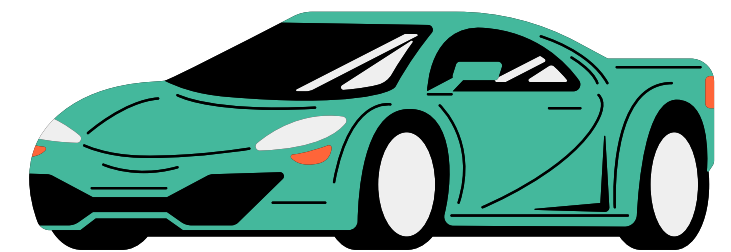
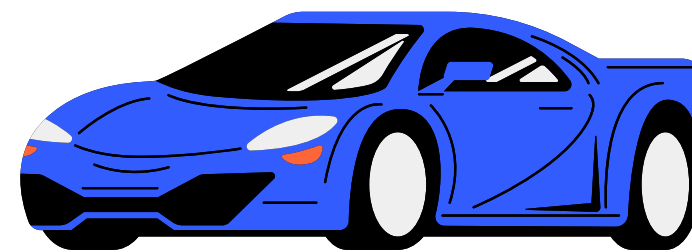
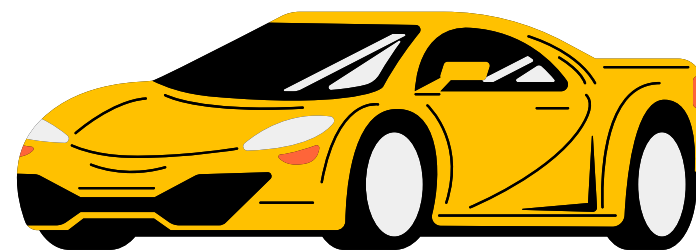
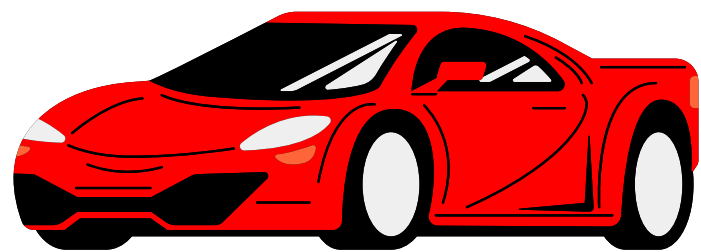
```
public class Main {  
    public static void main(String[] args) {  
        Person p = new Person();  
  
        p.name = "Alice"; → public, OK  
        p.address = "Bangkok"; → default,  
OK (same package)  
        p.setAge(25); → now OK, as setAge  
is public  
        System.out.println(p.getAge()); →  
prints 25  
    }  
}
```

Key takeaways:

- Setter must be public if you want to call it from another class
- Setter needs a type for the parameter
- Getter should be public and simply return the private value



FINAL CLASSES



WHAT'S A FINAL CLASS? RELATED TO FINAL EXAM?

- Final Class: A class marked as final cannot be inherited

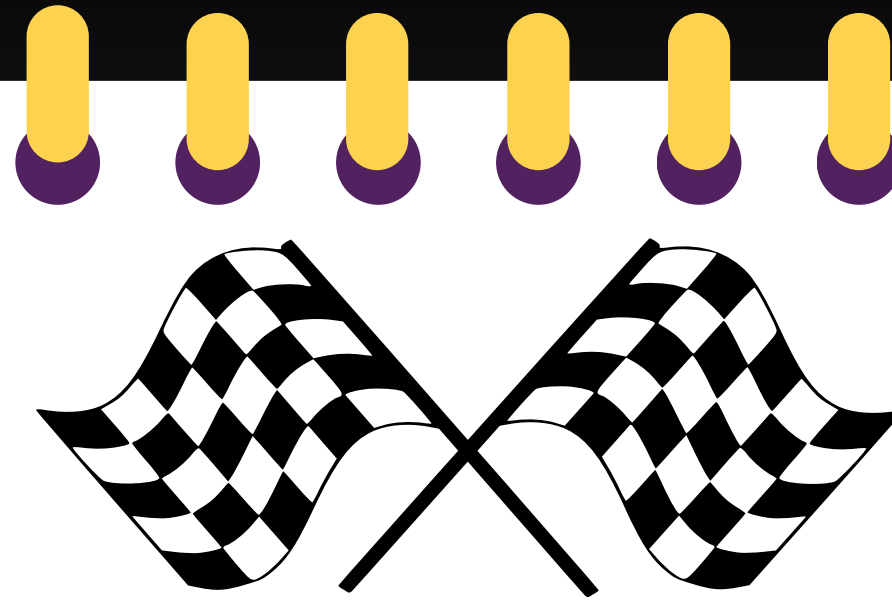
```
final class Animal {  
    // no class can extend Animal  
}
```

- Why Use final Classes?
 - Prevent modification of your class
 - Provide security and stability
 - Used when the class is complete and should not be changed

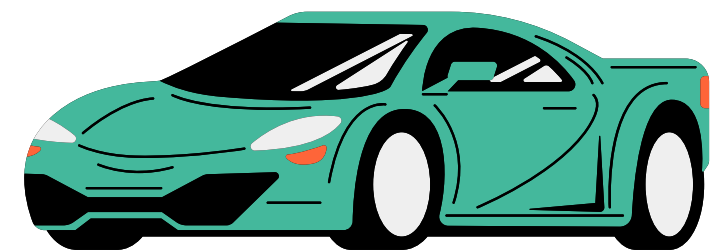
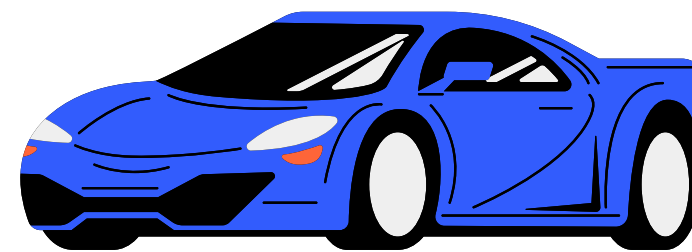
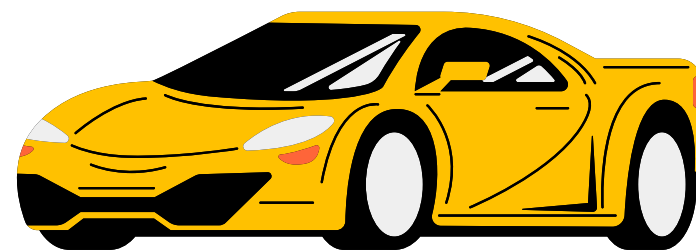
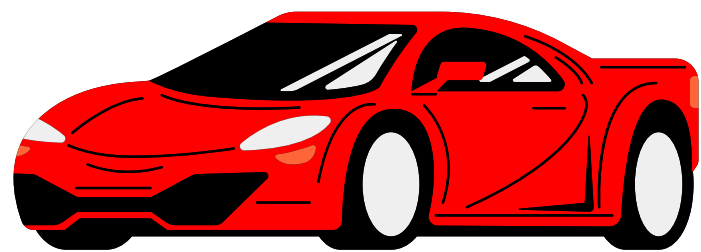
EXAMPLE OF FINAL CLASS

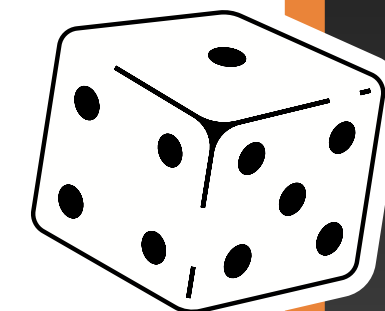
```
final class Calculator {  
    public int add(int a, int b) {  
        return a + b;  
    }  
}  
  
// class AdvancedCalc extends Calculator { }  
  
public class Main {  
    public static void main(String[] args) {  
        Calculator c = new Calculator();  
        System.out.println(c.add(5, 3)); → Output 8  
    }  
}
```

AdvancedCalc extends Calculator
because a final class can't be inherited



ANY QUESTIONS?





THANK YOU FOR
YOUR ATTENTION!

